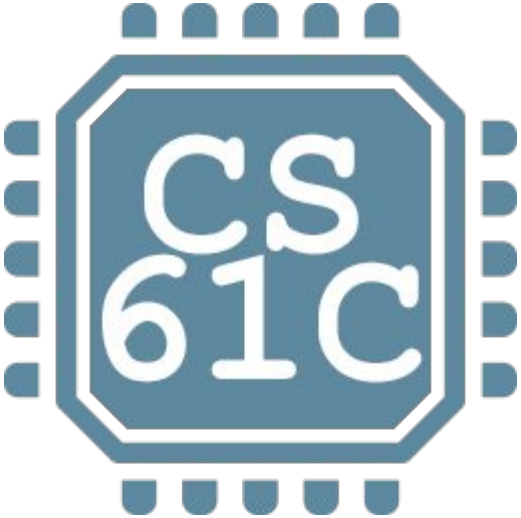
CS 61C Review for 161

...



61C Review





Important Note: This Review is not necessarily comprehensive of everything you will do in this course or will not replace the skills after having taken 61C

But we hope it helps you out move into this course <3

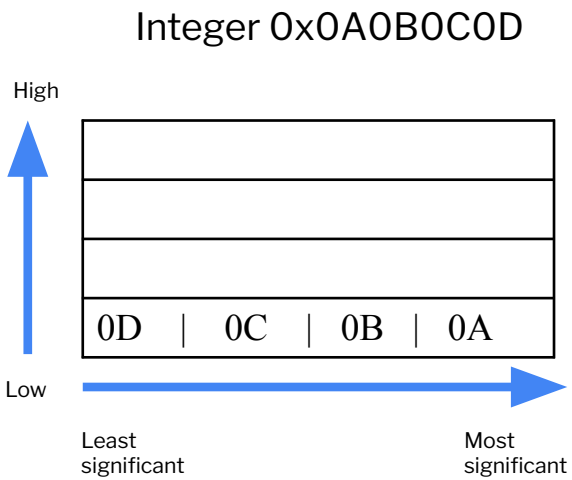
:-)

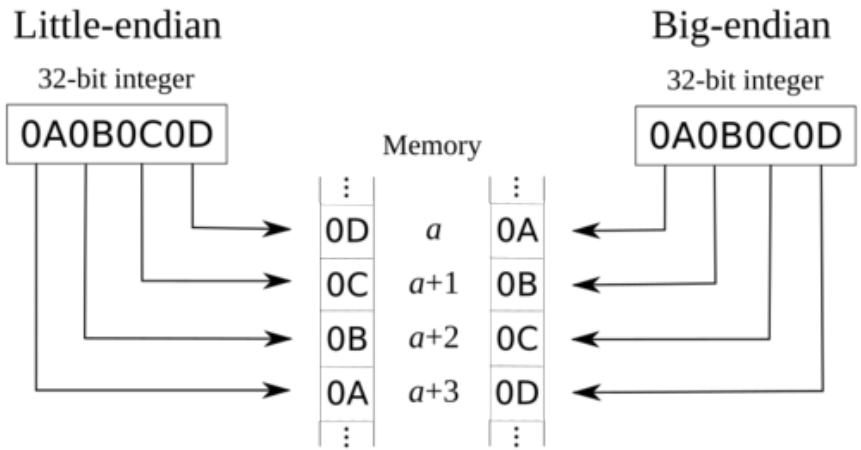


Endianness

Little Endian systems (x86): Least significant byte => lowest address in memory

Big Endian systems: Most significant byte => lowest address in memory





Write 0x0A0B0C0D in ___-endian starting at 0x67

Write 0x0A0B0C0D in ___-endian starting at 0x67

big

little

If big endian, write B+x in **ascending** order

If little endian, write B+x in **descending** order

B+0	B+1	B+2	B+3

B+3	B+2	B+1	B+0

Write 0x0A0B0C0D in ___-endian starting at 0x67

big

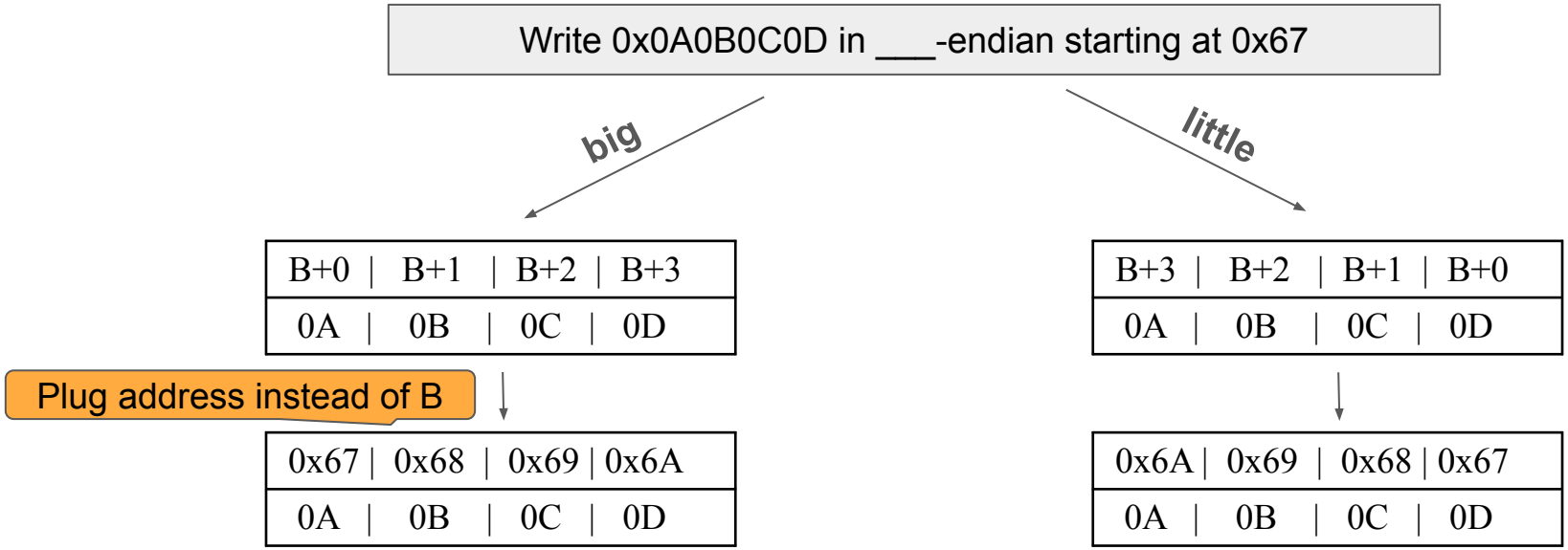
little

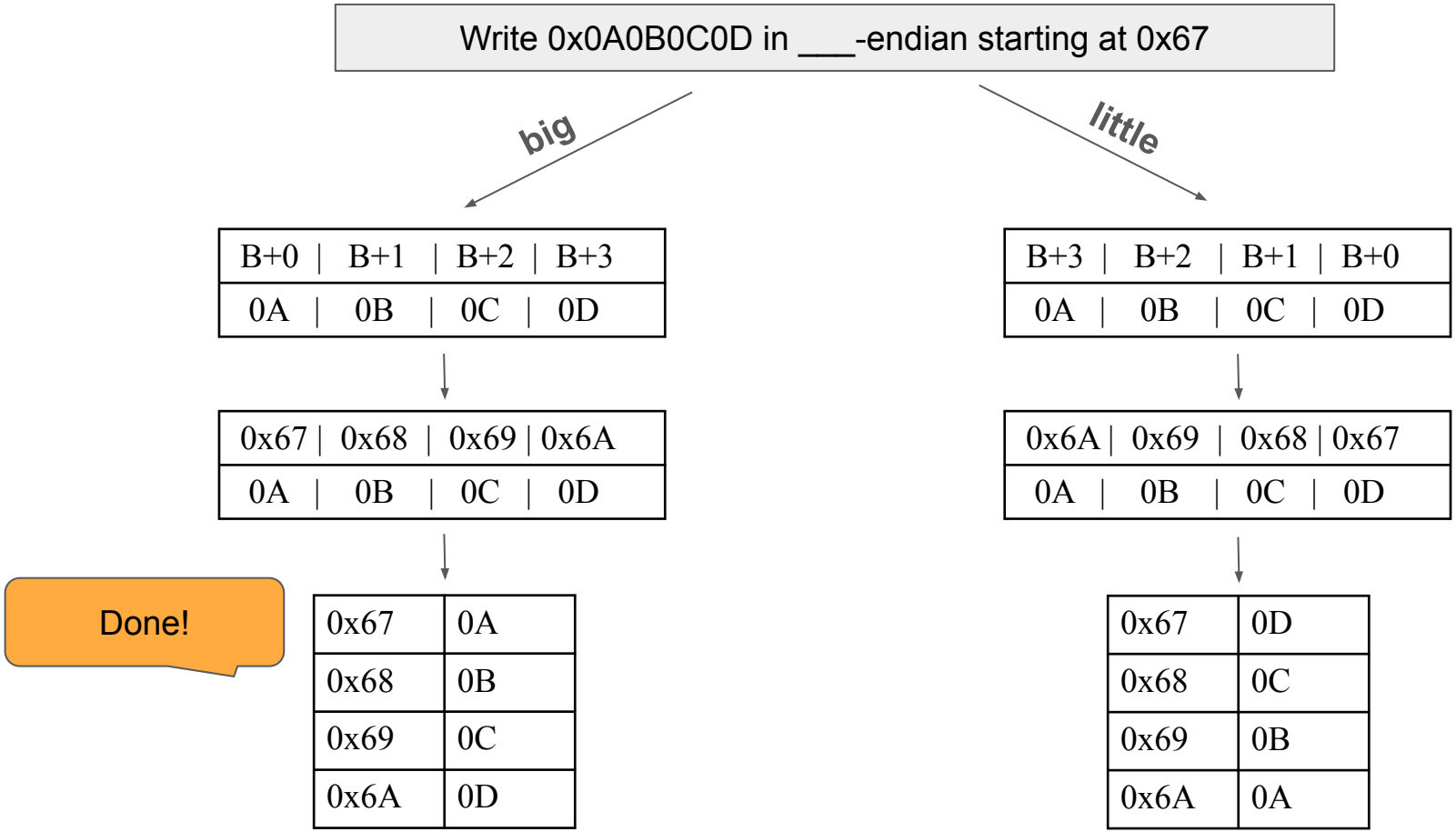
If big endian, write B+x in **ascending** order

If little endian, write B+x in **descending** order

B+0	B+1	B+2	B+3
0A	0B	0C	0D

B+3	B+2	B+1	B+0
0A	0B	0C	0D


Write 0x0A0B0C0D in ___-endian starting at 0x67
big
little
B+0 | B+1 | B+2 | B+3
0A | 0B | 0C | 0D
B+3 | B+2 | B+1 | B+0
0A | 0B | 0C | 0D
Plug address instead of B
0x67 | 0x68 | 0x69 | 0x6A
0A | 0B | 0C | 0D
0x6A | 0x69 | 0x68 | 0x67
0A | 0B | 0C | 0D


Write 0x0A0B0C0D in ___-endian starting at 0x67
big
little
B+0 | B+1 | B+2 | B+3
0A | 0B | 0C | 0D
B+3 | B+2 | B+1 | B+0
0A | 0B | 0C | 0D
0x67 | 0x68 | 0x69 | 0x6A
0A | 0B | 0C | 0D
0x6A | 0x69 | 0x68 | 0x67
0A | 0B | 0C | 0D
Done!
0x67 0A
0x68 0B
0x69 0C
0x6A 0D
0x67 0D
0x68 0C
0x69 0B
0x6A 0A

C is **statically typed** (i.e. types are known at compile time)

- This means you need to specify types when declaring variables (e.g. int x = 3;)

C is **weakly typed** (i.e. can cast between any types)

- Casts may also be implicit (e.g int to size_t)

Arrays are contiguous pieces of memory of the same type

- Denoted with `[]` (e.g. `int []` for an array of integers)
- **String** is an array of `chars` with last element being `null`

Pointers are references that hold the address of an object in memory.

- Essentially just unsigned integers.
- Prefix a pointer with * to return the value at the memory address that the pointer is holding (**"dereference"**)
- Prefix a variable with & to return the memory address of the variable.

(Adapted from CS162 slides)

Little Endian	+3	+2	+1	+0	
0x7FFFFFFC	00	00	00	08	`int a = 8`
0x7FFFFFF8	FF	FF	FF	FF	`int b = -1`
0x7FFFFFF4	7F	FF	FF	FC	`int *p = &a`
0x7FFFFFF0	27	CE	00	'O'	27, CE are garbage
0x7FFFFFEC	'T'	'N'	'I'	'P'	`char s[] = "PINTO"`

0x7FFFFFE8	00	00	00	02	`struct point {`
					`  int x;`
					`  int y;`
					`};`
					
					`struct point pt = {16, 2}`

0x7FFFFFE4	00	00	00	10	

Hexadecimal

A numerical system that represents numbers using a base of 16.

The numerals 0 to 9 are the same as in decimal, while 10 to 15 are represented with the letters “a” to “f”.

Pros:

- More readable than binary
- Easily converted to from binary

0x01 = 1 (unsigned)

0x0f = 15 (unsigned)

0xff = 255 (unsigned)

0xff = -1 (signed two’s complement)

$0x3ea4 = 3 * (16^3) + 14 * (16^2) + 10 * (16^1) + 4 * (16^0)$

$= 16036$

0x	3	e	a	4
=	$3 * (16^3)$	$14 * (16^2)$	$10 * (16^1)$	$4 * (16^0)$

Hexadecimal (cont.)

Hexadecimal to Binary Table

MATH MONKS

Hexadecimal (Base 16)	Binary (Base 2)	Hexadecimal (Base 16)	Binary (Base 2)
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Two’s Complement

Way of representing negative and positive integers using n-bits

- Smallest number: `0b1000...000` represents $-(2^{N-1})$.
- Largest number: `0b0111...111` represents $+(2^{N-1}-1)$

General properties:

1) Easy to do math with
2) Each bit sequence is an unique number
3) Most commonly used way to represent signed integers

Conversion

Two's complement → signed decimal number:

- If left-most bit is 0: Positive number
  - normal unsigned binary to integer conversion
- If left-most bit is 1: Negative number.
  - flip the bits
  - add 1.
  - convert to base-10
  - stick a negative sign in front.

Signed decimal number → Two's complement:

- If number is positive:
  - Normal conversion to base-2.
- If number is negative:
  - convert to base-2 as if unsigned
  - flip the bits
  - add 1.

Worksheet Question Time!

Question 1

Question 1.1 - 61C Review

What is the hexadecimal value of the decimal number 18?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.1 - 61C Review

What is the hexadecimal value of the decimal number 18?

0x12

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.2 - 61C Review

What is the value of `0x8339e833` + `0x20` in hexadecimal form?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.2 - 61C Review

What is the value of `0x8339e833` + `0x20` in hexadecimal form?

0x8339e853

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.3 - 61C Review

What is the value of `0x550ecdf2` + decimal 16 in hexadecimal form?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.3 - 61C Review

What is the value of `0x550ecdf2` + decimal 16 in hexadecimal form?

0x550ece02

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.4 - 61C Review

What is the largest **unsigned** 32-bit integer in hexadecimal format?

What is the result of adding 1 to that number?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.4 - 61C Review

What is the largest unsigned 32-bit integer in hexadecimal format?

0xffffffff

What is the result of adding 1 to that number?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.4 - 61C Review

What is the largest unsigned 32-bit integer in hexadecimal format?

0xffffffff

What is the result of adding 1 to that number?

0x00000000

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.5 - 61C Review

What is the largest positive **signed** 32-bit integer in hexadecimal format?

What is the result of adding 1 to that number?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.5 - 61C Review

What is the largest positive **signed** 32-bit integer in hexadecimal format?

0x7fffffff

What is the result of adding 1 to that number?

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.5 - 61C Review

What is the largest positive **signed** 32-bit integer in hexadecimal format?

0x7fffffff

What is the result of adding 1 to that number?

0x80000000

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Question 1.6 - 61C Review

If you interpret an n-bit two’s complement number as an unsigned number, would the negative numbers be smaller or larger than positive numbers?

Question 1.6 - 61C Review

If you interpret an n-bit two's complement number as an unsigned number, would the negative numbers be smaller or larger than positive numbers?

Larger

Question 1.6 - 61C Review

If you interpret an n-bit two's complement number as an unsigned number, would the negative numbers be smaller or larger than positive numbers?

Larger

The negative numbers in two's complements all have their most significant bit (MSB) set to 1 and all the positive numbers in two's complements have their MSB set to 0.

Thus, if we treat them as unsigned integers, the negatives will be interpreted as larger than the positives.

Question 1.7 - 61C Review

How many bytes are needed to represent `char[16]`?

Question 1.7 - 61C Review

How many bytes are needed to represent `char[16]`?

16 bytes

Question 1.7 - 61C Review

How many bytes are needed to represent `char[16]`?

16 bytes

A `char[16]` is an array of 16 characters. Each character requires one byte of memory to store. Thus, in total we will need 16 bytes to store all the characters in the array.

Question 1.8 - 61C Review

How many bytes are needed to represent `int[8]`?

Question 1.8 - 61C Review

How many bytes are needed to represent `int[8]`?

32 bytes

Question 1.8 - 61C Review

How many bytes are needed to represent `int[8]`?

32 bytes

An `int[8]` is an array of 8 integers. Each integer requires **four bytes** of memory to store. Thus, in total we will need 8 * 4 = 32 bytes to store all the integers in the array.

Question 1.9 - 61C Review

In a **little-endian 32-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.9 - 61C Review

In a **little-endian 32-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

(unused)	(unused)	(unused)	(unused)
0xEF	0xBE	0xAD	0xDE

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.9 - 61C Review

In a **little-endian 32-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

(unused)	(unused)	(unused)	(unused)
0xEF	0xBE	0xAD	0xDE

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Recall: Due to the system being little-endian, the least significant byte of a value (such as a pointer) will be placed at the lowest memory location.

Question 1.10 - 61C Review

In a **little-endian 64-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.10 - 61C Review

In a **little-endian 64-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

`0x00`	`0x00`	`0x00`	`0x00`
`0xEF`	`0xBE`	`0xAD`	`0XDE`

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.10 - 61C Review

In a **little-endian 64-bit system**, how would you represent the pointer `0xDEADBEEF`?

Fill in the following table accordingly.

0x00	0x00	0x00	0x00
0xEF	0xBE	0xAD	0XDE

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Recall: Due to the system being a 64-bit system, pointers will require 8 bytes to represent.

Question 1.11 - 61C Review

In a **little-endian 32-bit system**, how would you represent the char array ”`ABCDEFGH`”?

Fill in the following table accordingly.

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.11 - 61C Review

In a **little-endian 32-bit system**, how would you represent the char array ”`ABCDEFGH`”?

Fill in the following table accordingly.

`E`	`F`	`G`	`H`
`A`	`B`	`C`	`D`

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

Question 1.11 - 61C Review

In a **little-endian 32-bit system**, how would you represent the char array ”ABCDEFGH”?

Fill in the following table accordingly.

E	F	G	H
A	B	C	D

For the following subparts, assume each block is 1 byte, and addresses increase from left-to-right and bottom-to-top.

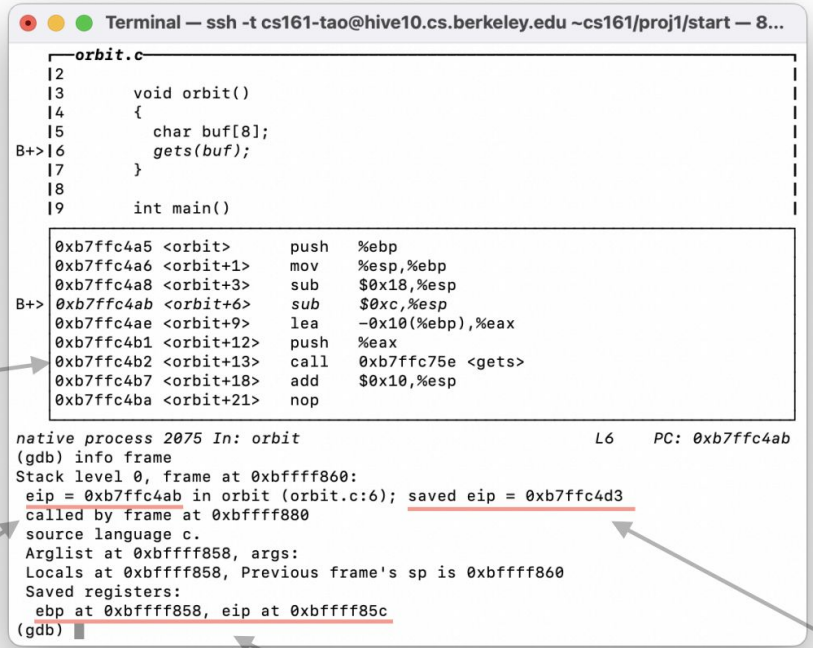
Recall: For arrays, the first element will always be at the lowest memory address.

GDB Common Commands

Commands	Abbreviation	Description
start	start	begin running the program and stop at line 1 in main
***step**	**s**	execute the current line of code (this command will step into functions)
***next**	**n**	execute the current line of code (this command will not step into functions)
finish	fin	executes the remainder of the current function and returns to the calling function
***print**	**p**	prints the value of the argument
***break [line #]**	**b [#]**	set a breakpoint at the specified location, use filename.c:linenum to set a breakpoint in a specific file
run	r	execute the program until termination or reaching a breakpoint
continue	c	continues the execution of a program that was paused
***backtrace**	**bt**	print one line per frame for frames in the stack
quit	q	exits gdb

"Info Frame" in GDB

The "B" indicates that we have a breakpoint set at this line number. The arrow ">" points to the next line that the CPU will execute.



If things look a little wonky for whatever reason (e.g. text is out of place, or you resized the GDB window), use the **refresh** command to re-render the screen!

The addresses that you see here are the addresses that these instructions reside at. Remember, this is in the **code** section of memory.

This is the **instruction pointer (EIP)** - it points to the address of the next instruction to be executed.

These values indicate the **addresses** of the SFP and RIP, respectively. We often use the address of the RIP to do our buffer overflow math.

This is the **value** of the **RIP of the current frame**. When this function completes, the CPU will jump to the instruction at stored at this address.

printf

`printf("text here, number: %d, string: %s", num, word)` ← num is an integer variable and word is a pointer to a char array ending in the null terminator '\0'

The function printf prints out an input string formatted based on specifiers within the string such as %d or %x and pointers input to use as inputs to use for those specifiers.

Specifier	Description
%d	prints a signed integer obtained from the corresponding input variable
%u	prints an unsigned integer obtained from the corresponding input variable
%x	prints an unsigned hexadecimal integer obtained from the corresponding input variable
%c	prints a single character obtained from the corresponding input variable
%s	prints a char array obtained starting from the address of the corresponding input variable, stopping at but not including the null terminator
%n	does not modify the printed output, takes the number of characters written so far and writes this sum to the input variable

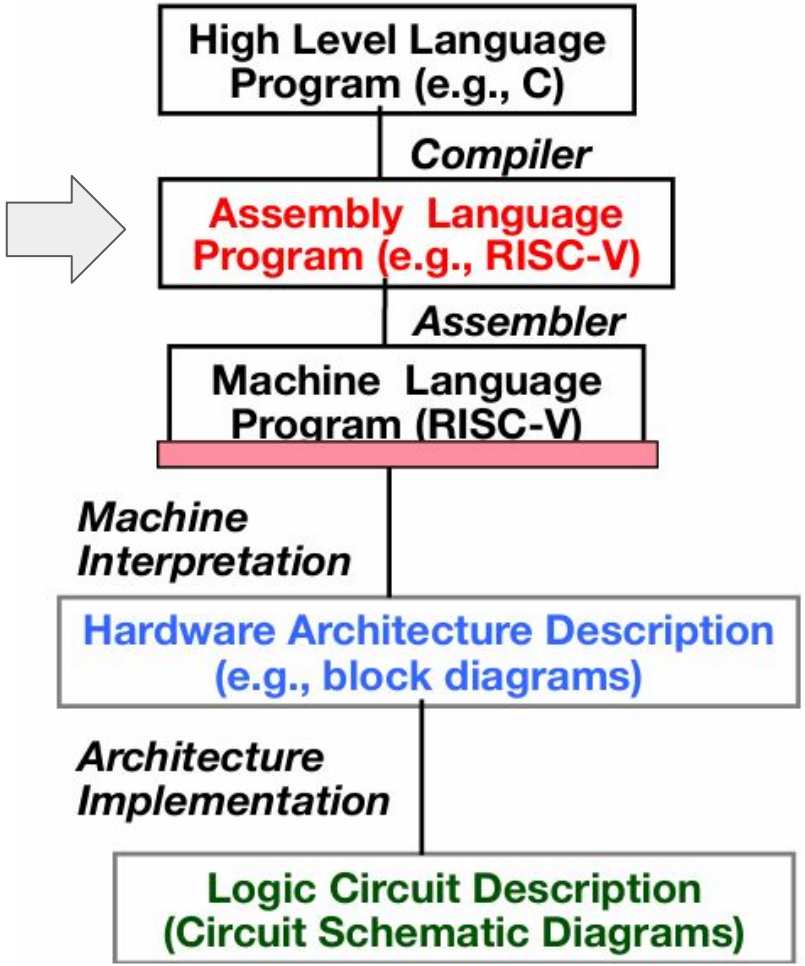
61C Review & x86

x86 Architecture

x86 is an **assembly language**, like RISC-V.

That means many of the concepts we learned in CS 61C (memory layout, calling convention, etc.) will show up in some form.

We need to know how x86 works before we can find any security issues in codes!



Overview

Conceptually, a **process** is an instance of a program running.

The **CPU** is hardware that executes and runs code for a program. It uses two distinct things for executing a program.

Memory: tracks program data/variables.

Registers: tracks locations of things (e.g. stack pointer, instruction pointer).

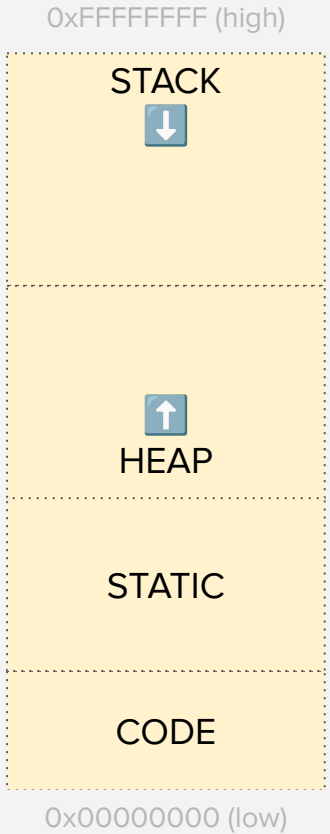


Memory

The **Code** section lives at the bottom of our address space and contains machine code (x86 instructions).

The **Static** section contains any static variables and constants declared outside functions.

The **Heap** section is dynamically allocated whenever our code calls `malloc()` - it starts at lower addresses and grows up.

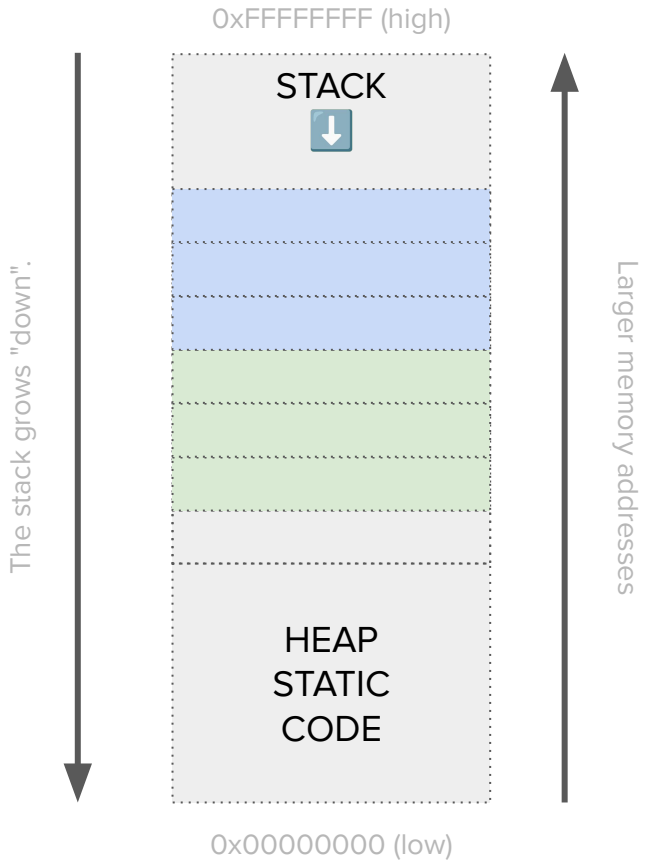


The Stack

Any time our code calls a function, space is allocated on the **stack** for local variables. That space goes away when a function returns.

Each row of the diagram to the right is **one word = 4 bytes = 32 bits**.

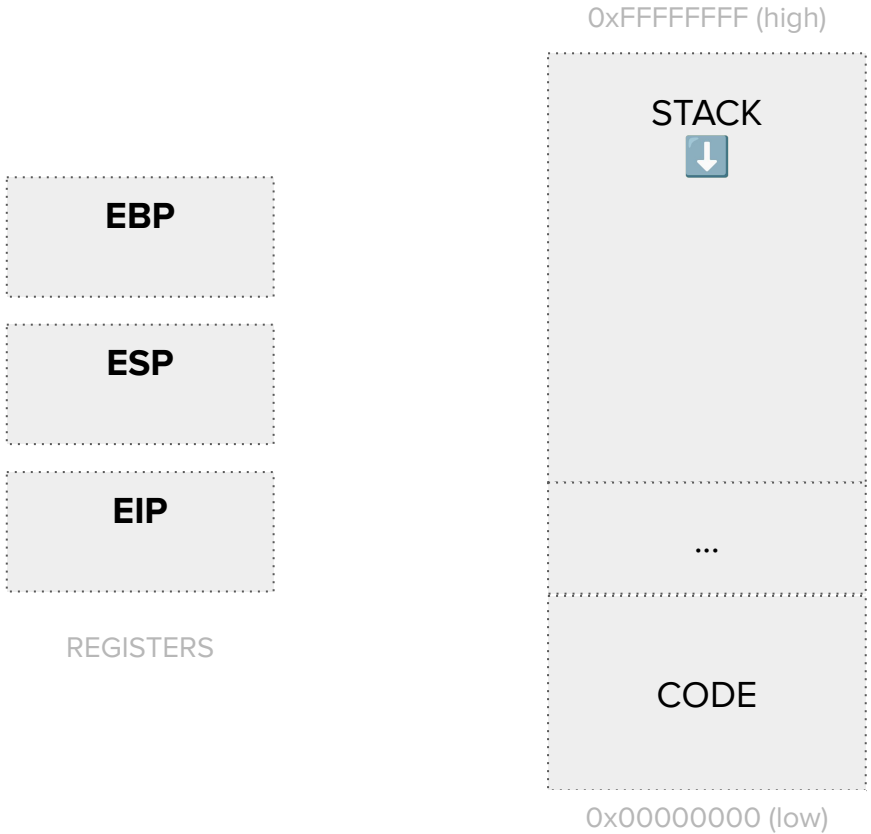
A **stack frame** can be thought of as the local scope of a function. It corresponds directly to a function being called.



Registers

The CPU needs a way to keep track of the top/bottom of a stack frame. For this, it uses **registers**.

Registers are 32-bit "boxes" located directly on the CPU. Since memory addresses are also 32-bit, they can point to locations in memory.

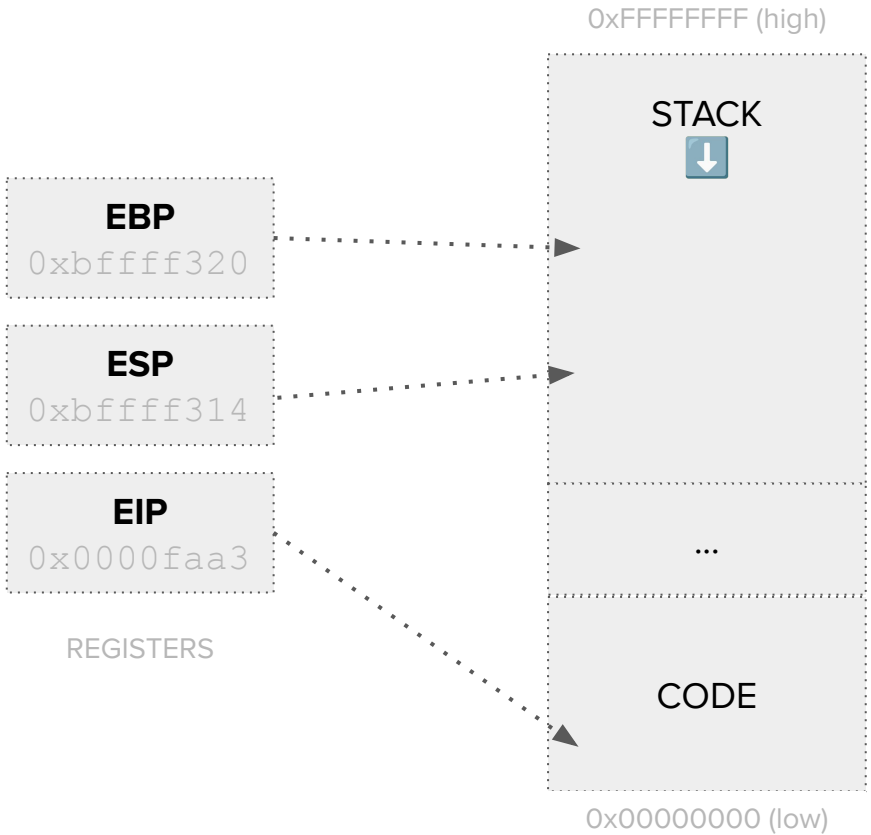


Registers

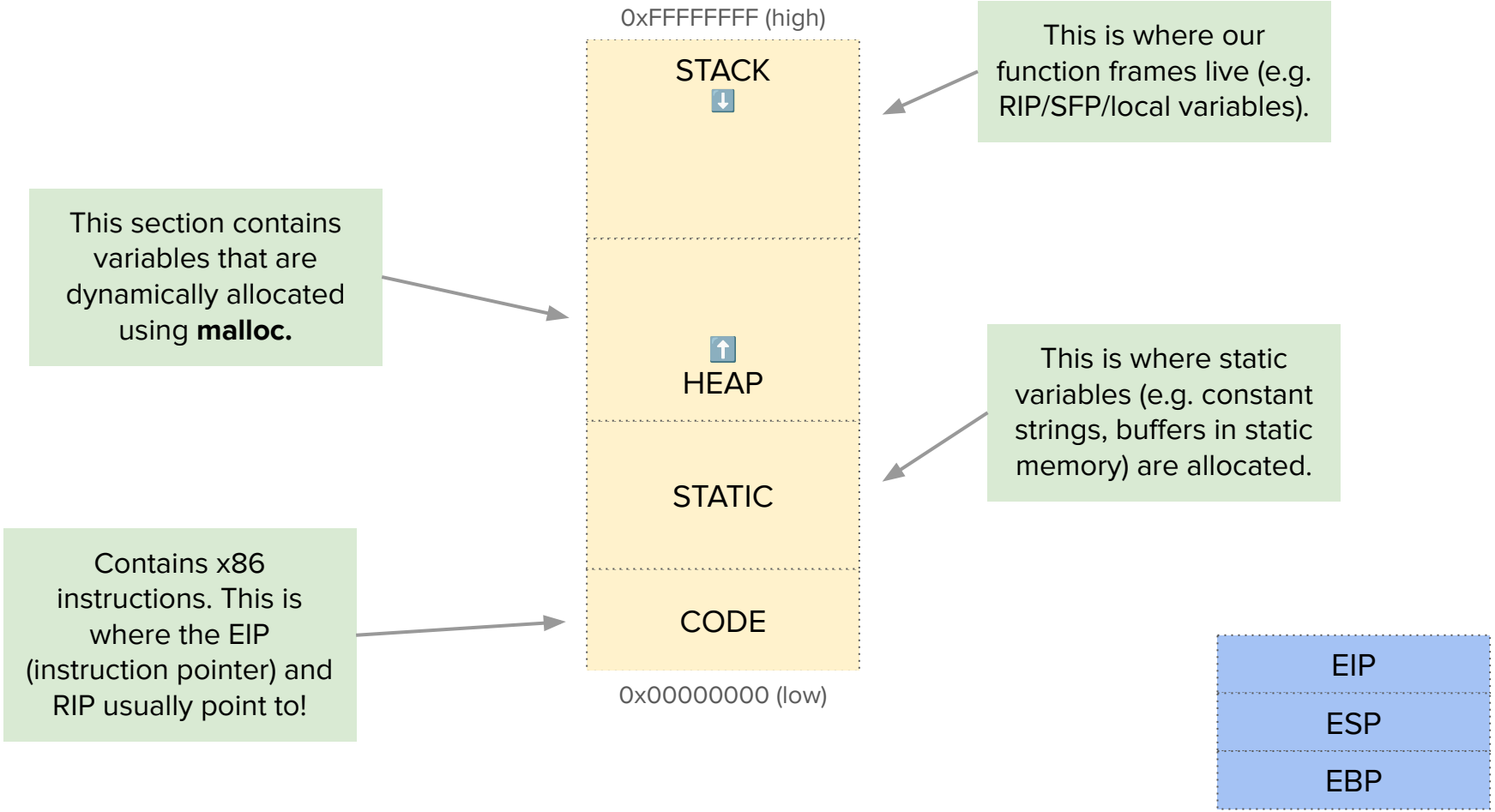
The **EBP** points to the top of the current stack frame.

The **ESP** points to the bottom of the current stack frame. When we push values onto the stack, the ESP moves down.

The **EIP** points to the current instruction.



Calling Convention


0xFFFFFFFF (high)
STACK
⬇️
This is where our function frames live (e.g. RIP/SFP/local variables).
This section contains variables that are dynamically allocated using **malloc.**
⬆️
HEAP
This is where static variables (e.g. constant strings, buffers in static memory) are allocated.
STATIC
Contains x86 instructions. This is where the EIP (instruction pointer) and RIP usually point to!
CODE
0x00000000 (low)
EIP
ESP
EBP

Function Calls

When we make a function call, these **register values** need to change!

We also need a way of **saving the information we need** to return back to the original function.

These steps are defined by "**calling convention**".

```
void foo(int a, int b)
{
    int bar[2];
    bar[0] = a;
    bar[1] = b;
}

int main(void)
{
    foo(0x11, 0x22);
    return 0;
}
```

Calling Convention

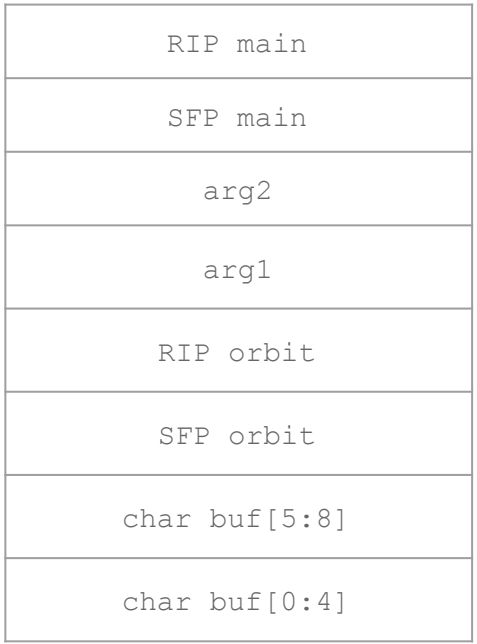
1. Push arguments onto stack (in reverse order).
2. Push old EIP (RIP) onto stack.
3. Move EIP.
4. Push old EBP (SFP) onto stack.
5. Move EBP.
6. Move ESP.
7. Execute the function.
8. Move ESP.
9. Restore old EBP (SFP).
10. Restore old EIP (RIP).
11. Remove arguments from stack.

```
void foo(int a, int b)
{
    int bar[2];
    bar[0] = a;
    bar[1] = b;
}

int main(void)
{
    foo(0x11, 0x22);
    return 0;
}
```

A normal x86 stack frame

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```

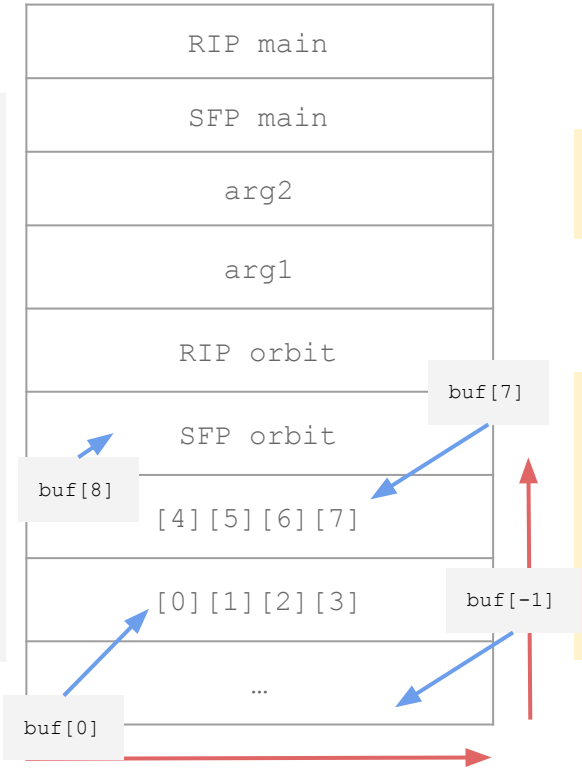


A stack diagram for the code to the left if we set a breakpoint at Line 3.

Tip #1: know how to draw stack frames!

A normal x86 stack frame

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```



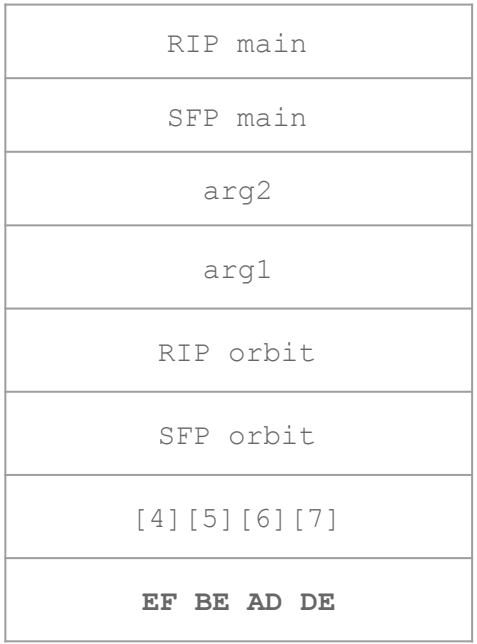
Tip #2: Know how values are written to a buffer on a stack.

Remember that we write **left-to-right** and **bottom-upwards** when writing to a buffer.

In this case, buf[0] is at the lowest memory address, and buf[7] is at the highest address.

A normal x86 stack frame

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```




```
print("\xEF\xBE\xAD\xDE")

address stored on stack:
0xDEADBEEF
```

Tip #3: Know how addresses are written to & stored on the stack!

x86 is little-endian, which means the **least significant byte** is stored at the **smallest memory address**.

A normal x86 stack frame

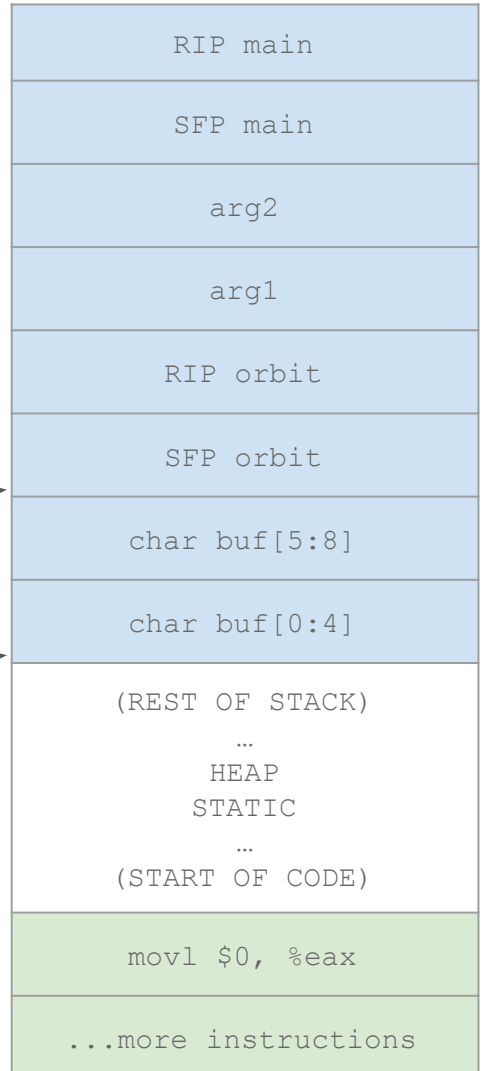
```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```

Tip #4: Know where the important registers point to during normal execution of a function.

EBP (base pointer)
e.g. 0xffffffab8

ESP (stack pointer)
e.g. 0xffffffab0

EIP (instruction pointer)
e.g. 0x0000abcd



Calling Convention

1. Push arguments onto stack (in reverse order).
2. Push old EIP onto stack (RIP).
3. Move EIP.

main

4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. Execute the function.
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.

orbit

11. Remove arguments from stack.

main

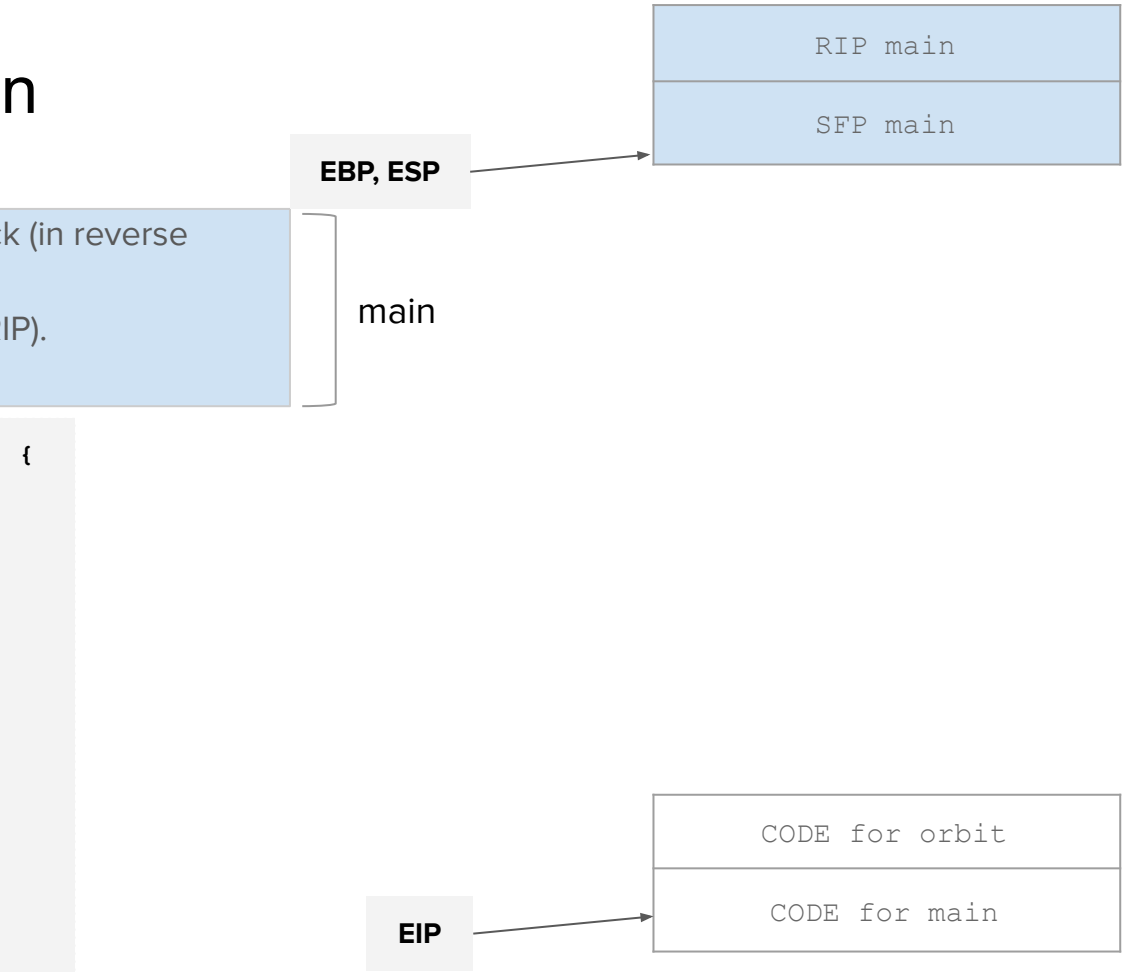
```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```

Calling Convention

1. Push arguments onto stack (in reverse order).
2. Push old EIP onto stack (RIP).
3. Move EIP.

main

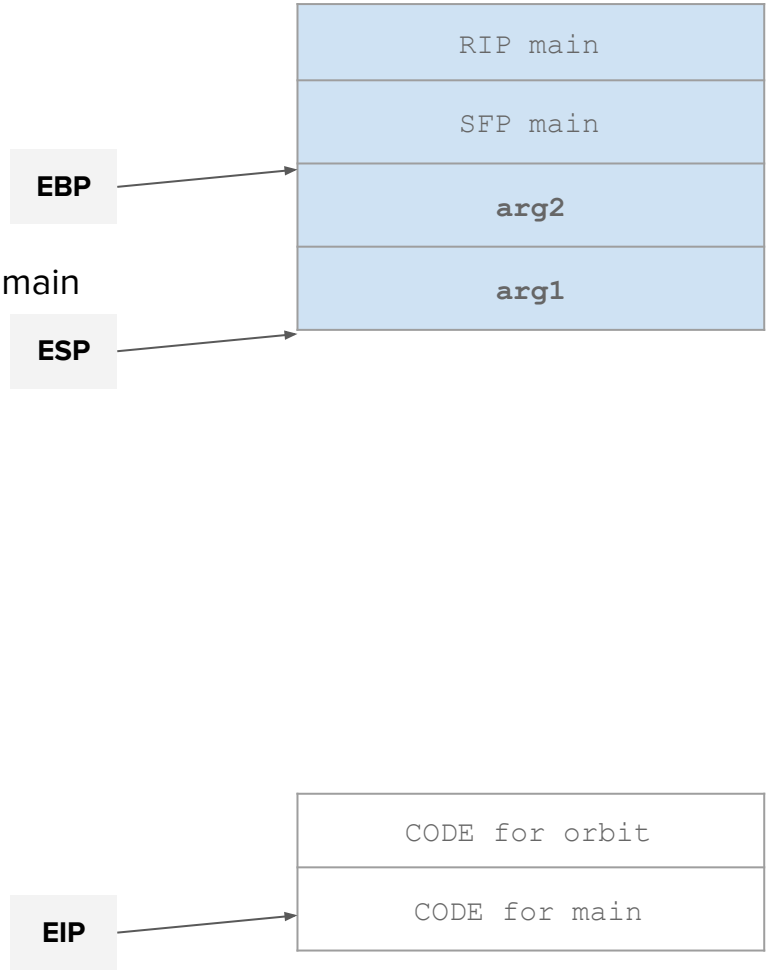
```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```



Calling Convention

1. **Push arguments onto stack (in reverse order).**
2. Push old EIP onto stack (RIP).
3. Move EIP.

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```

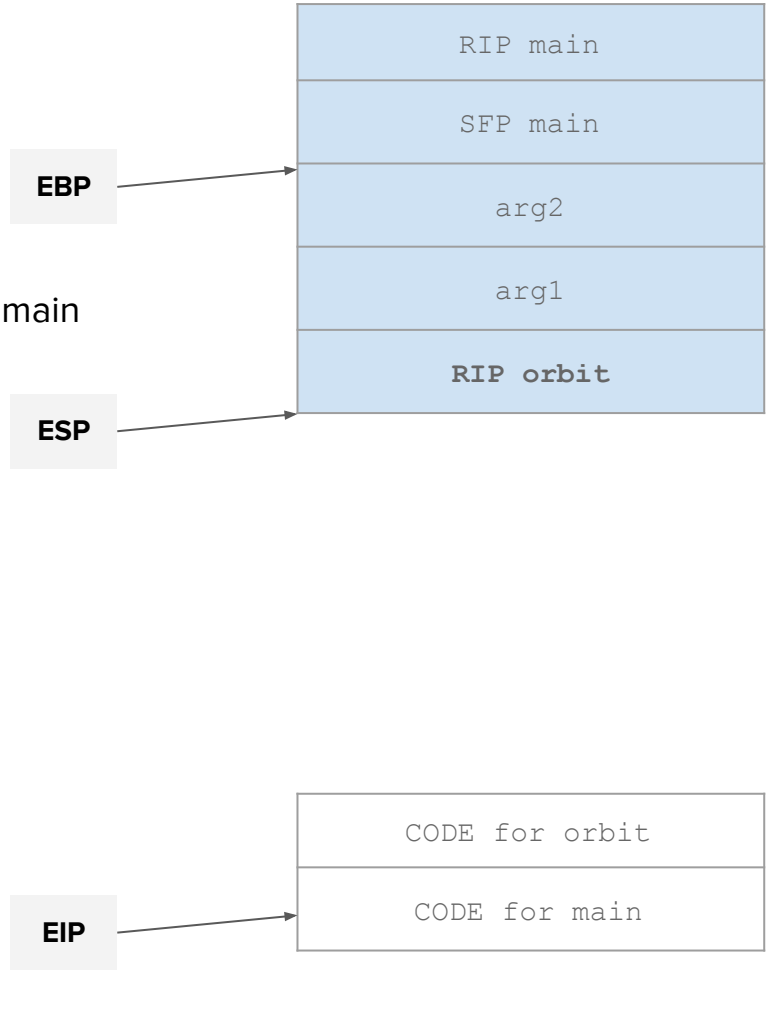


Calling Convention

1. Push arguments onto stack (in reverse order).
2. **Push old EIP onto stack (RIP).**
3. Move EIP.

main

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```

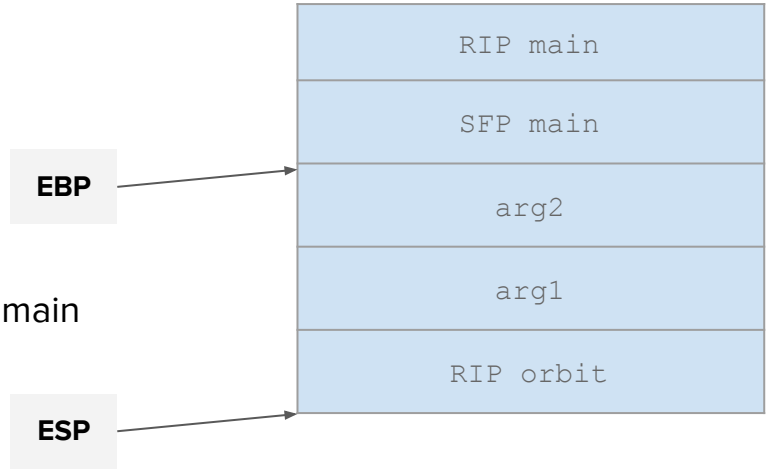


Calling Convention

1. Push arguments onto stack (in reverse order).
2. Push old EIP onto stack (RIP).
3. **Move EIP.**

main

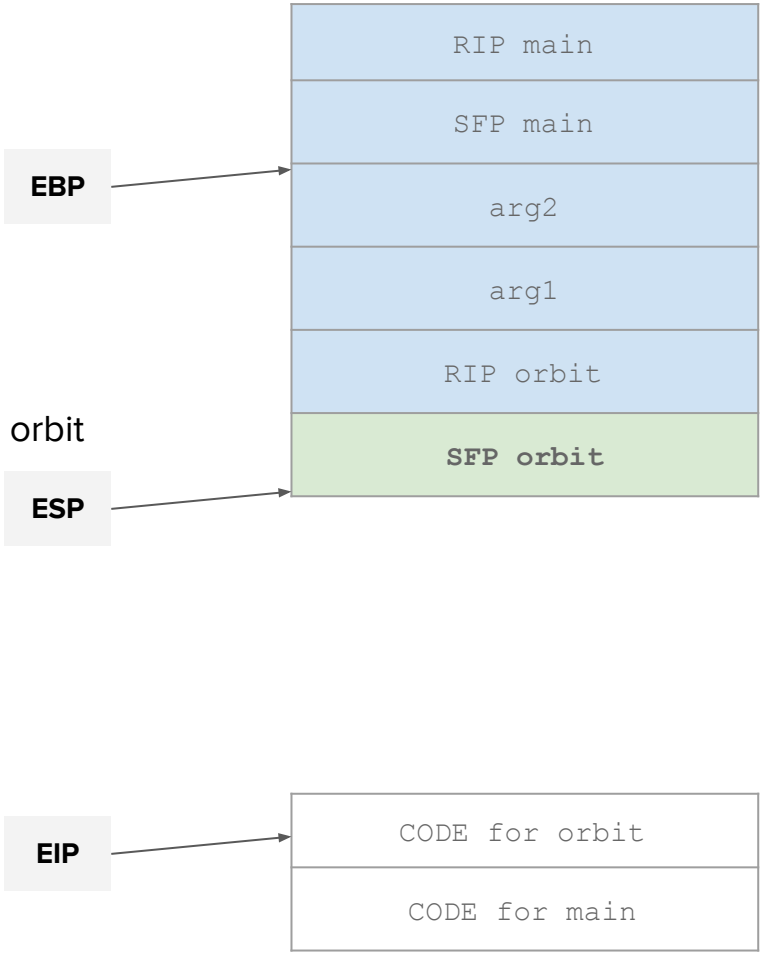
```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```





Calling Convention

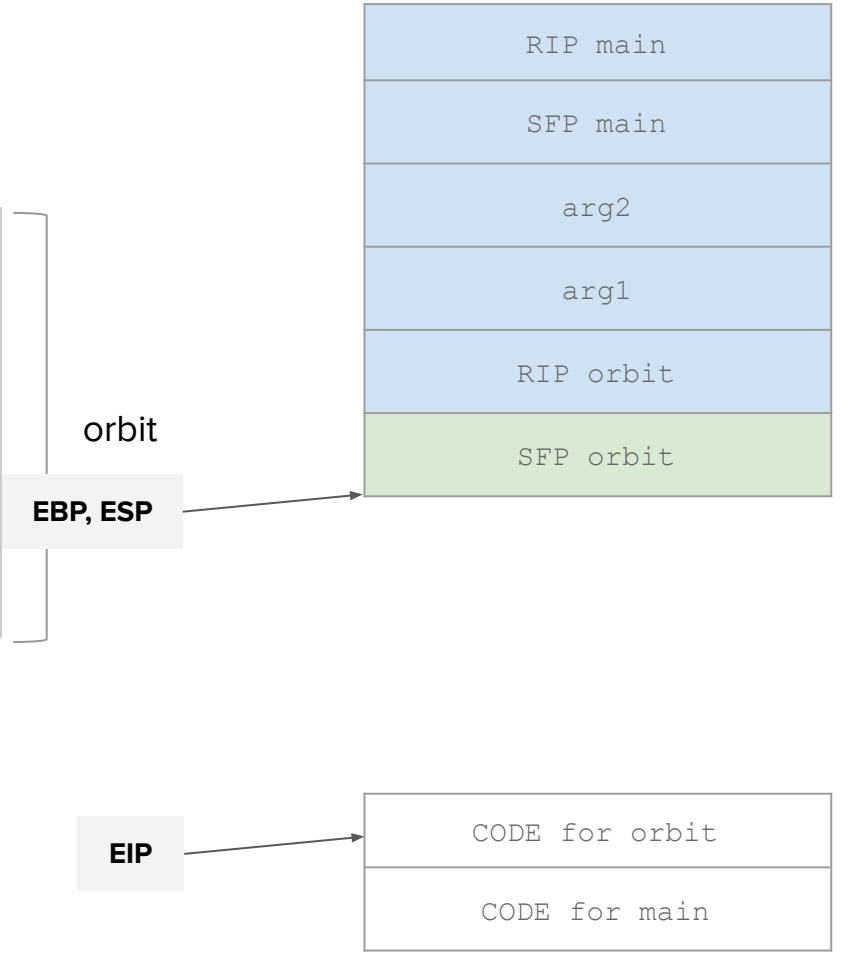
4. **Push old EBP onto stack (SFP).**
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. Execute the function.
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.




```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```

Calling Convention

4. Push old EBP onto stack (SFP).
5. **Move EBP down to ESP.**
6. Move ESP down for new frame.
7. Execute the function.
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.

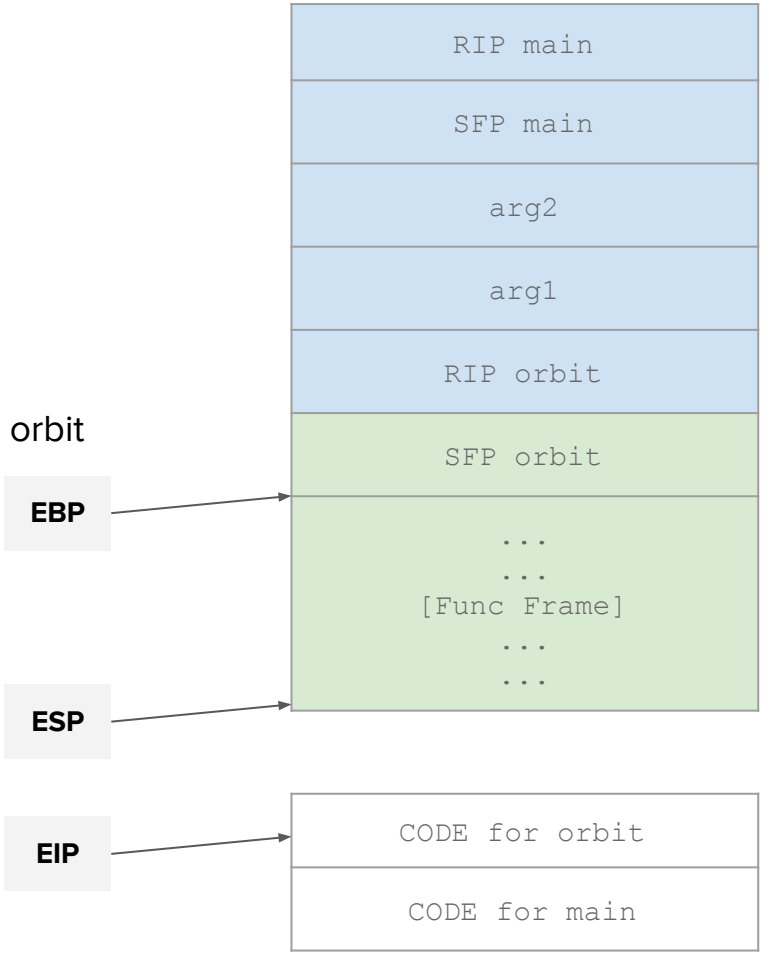



```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```

Calling Convention

4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. **Move ESP down for new frame.**
7. Execute the function.
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.

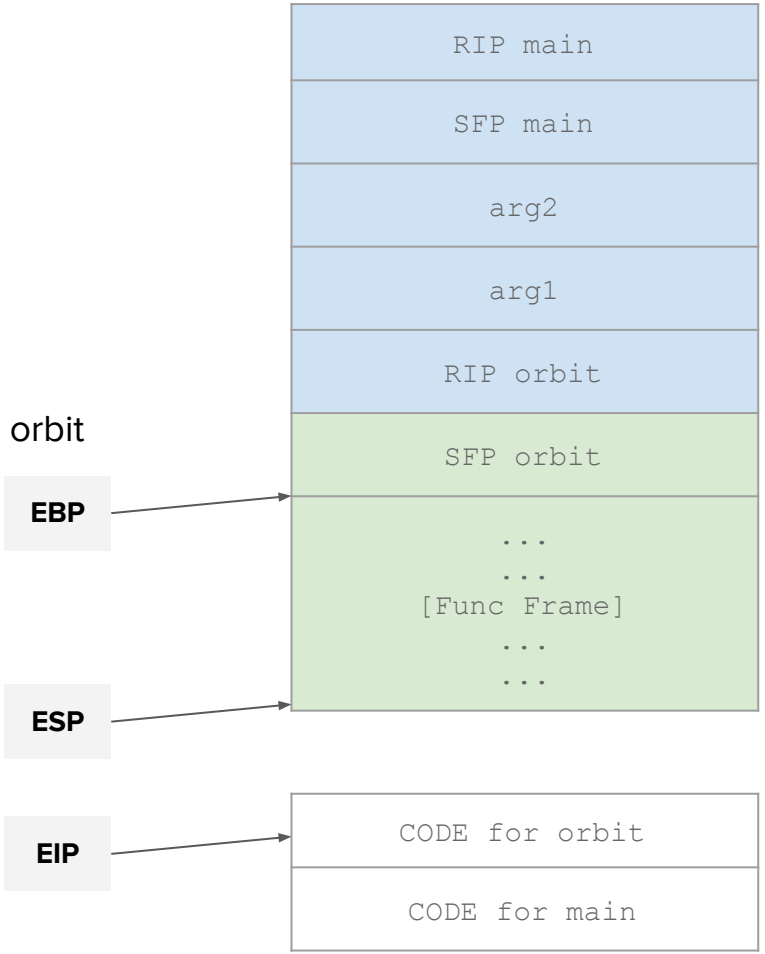
```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```



Calling Convention

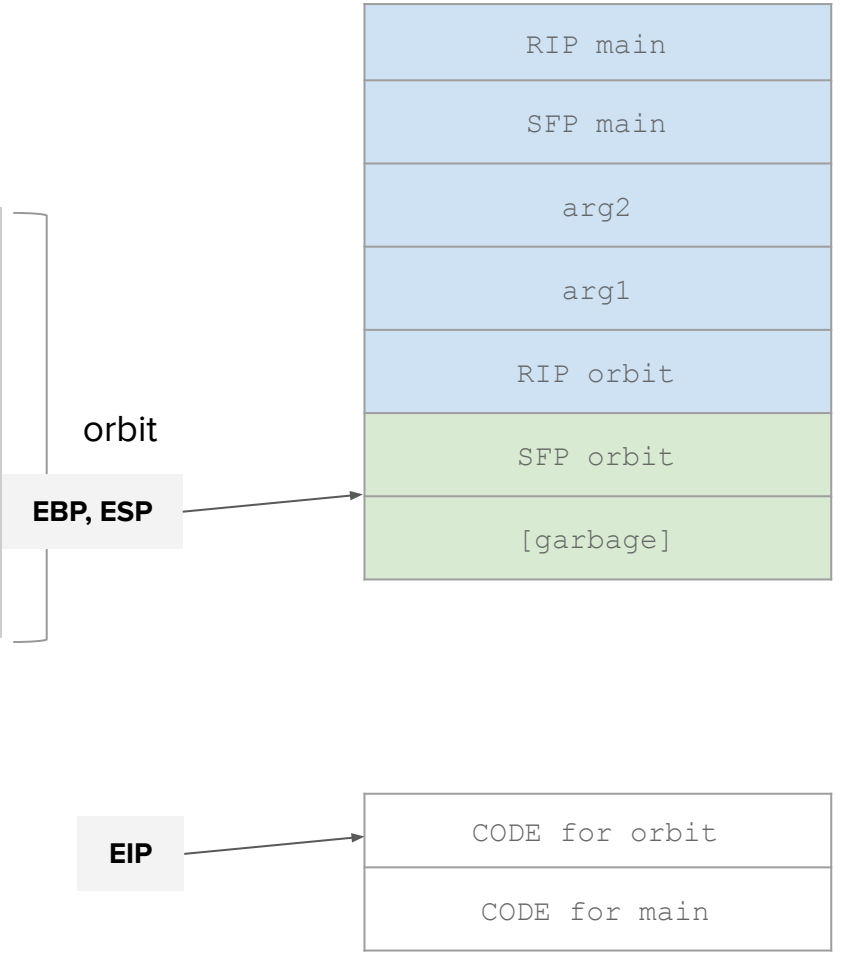
4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. **Execute the function.** 🤑
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```



Calling Convention

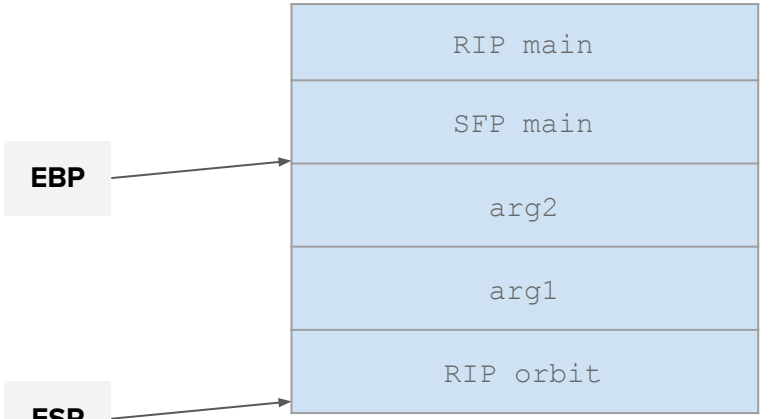
4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. Execute the function.
8. **Move ESP up to EBP.**
9. Restore old EBP by popping SFP.
10. Restore old EIP by popping RIP.




```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```

Calling Convention

4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. Execute the function.
8. Move ESP up to EBP.
9. **Restore old EBP by popping SFP.**
10. Restore old EIP by popping RIP.



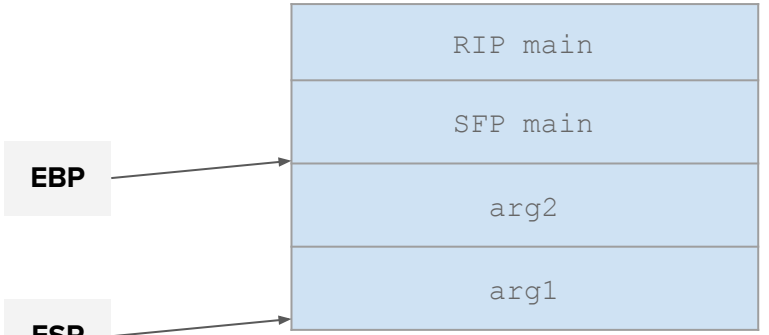

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```



Calling Convention

4. Push old EBP onto stack (SFP).
5. Move EBP down to ESP.
6. Move ESP down for new frame.
7. Execute the function.
8. Move ESP up to EBP.
9. Restore old EBP by popping SFP.
10. **Restore old EIP by popping RIP.**

orbit



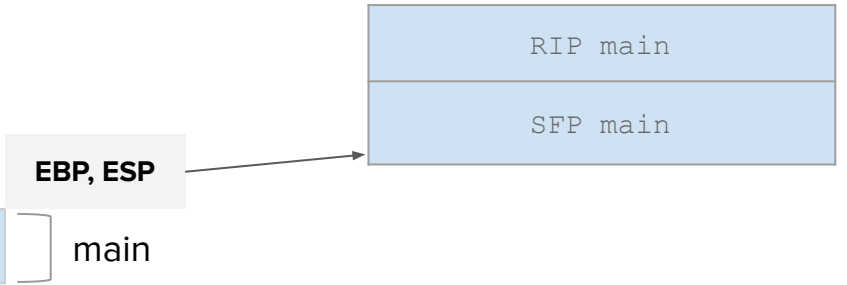

```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
```



Calling Convention

11. **Remove arguments from stack.**

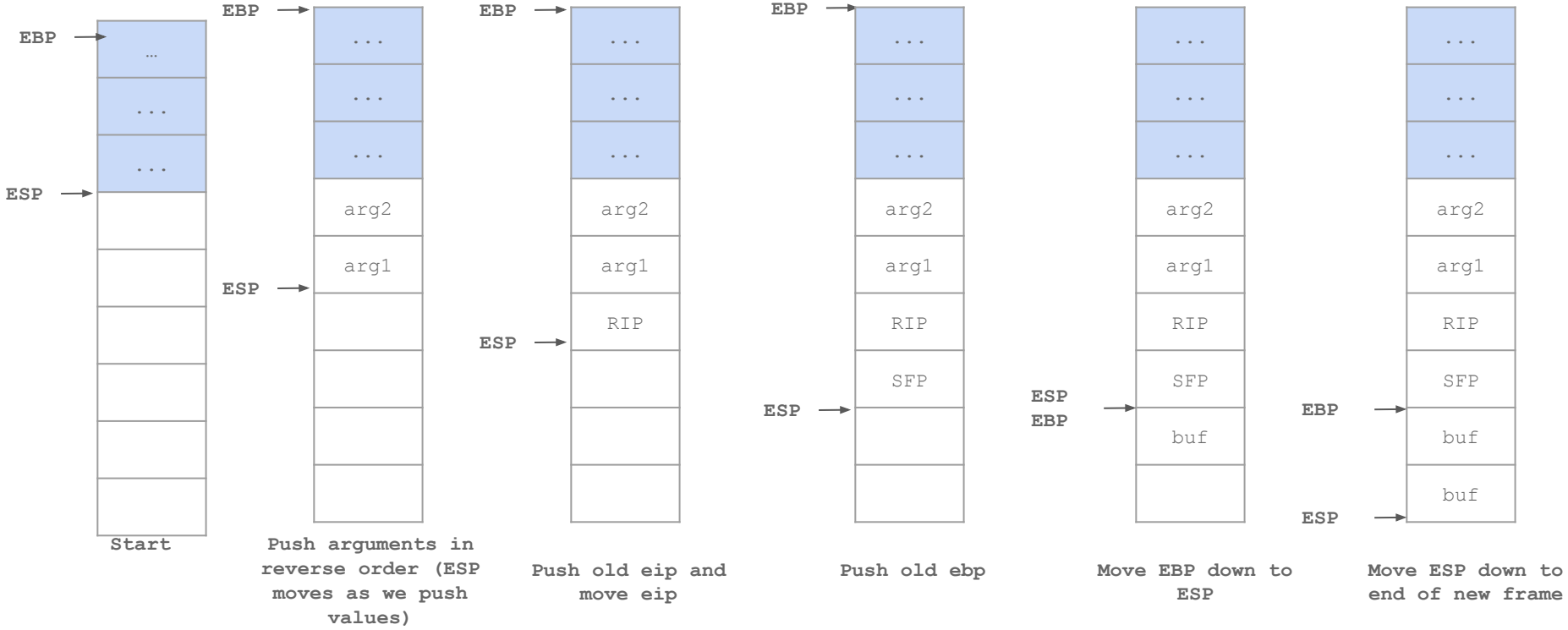
main




```
1. void orbit(int arg1, int arg2) {
2.     char buf[8];
3.     gets(buf);
4. }
5. int main(void) {
6.     orbit(1, 2);
7.     return;
8. }
```



A normal function prologue…



EIP → CODE [throughout this execution, the EIP points down here at x86 instructions].

A normal function epilogue…

